INT 422
Pattern Recognition
2024

Dr. Maysa Khalil Noby
Lecturer of Information Technology & Computer Science
Maysa.khalil@su.edu.eg

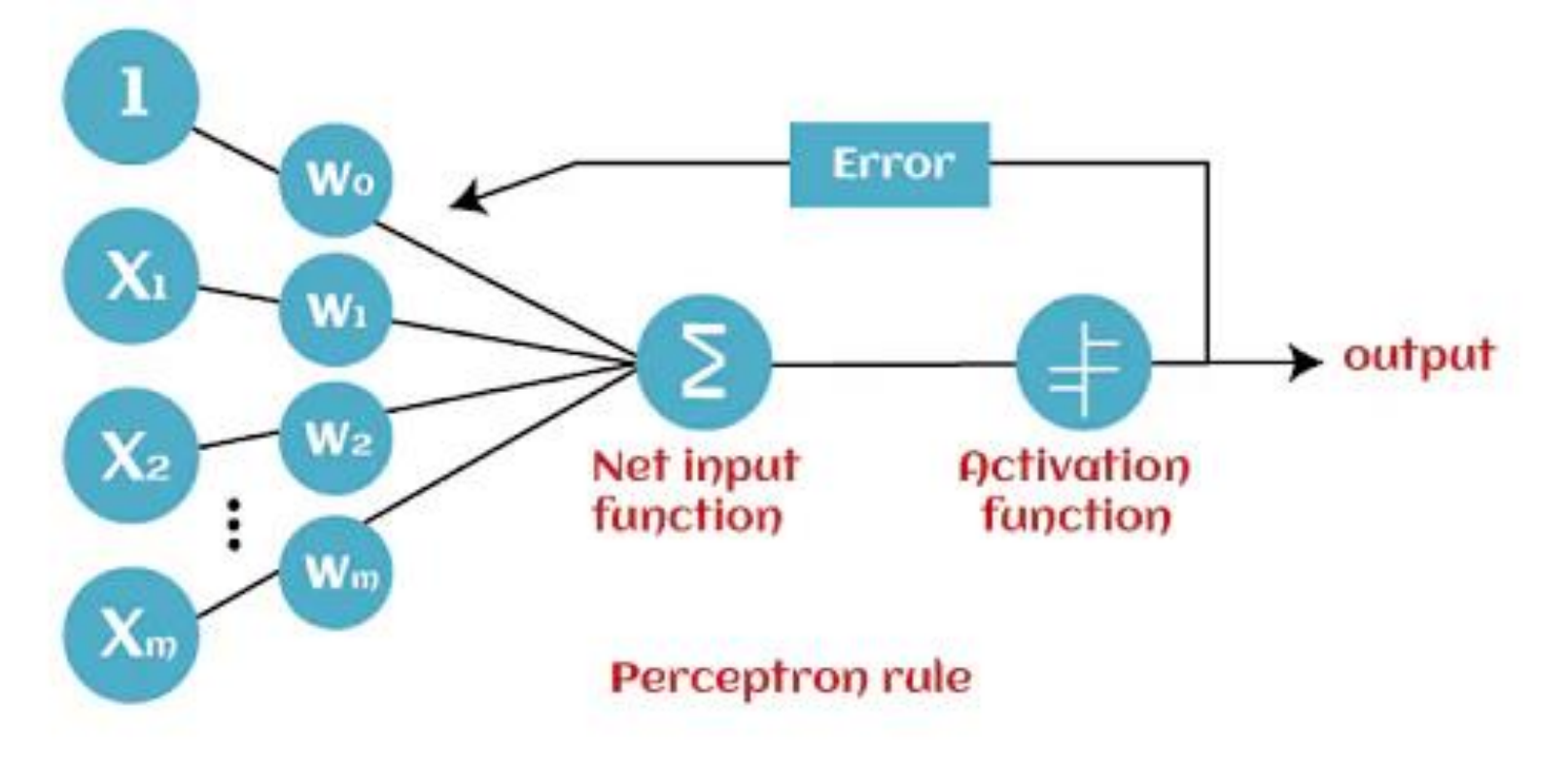
Lecture 7-8: Agenda

- **Introduction**
- **How Perceptron Works**
- **Perceptron Learning Algorithm**
- **Delta Rule Learning Algorithm**
- **Examples**
- **MLNN**

Perceptron (imp)

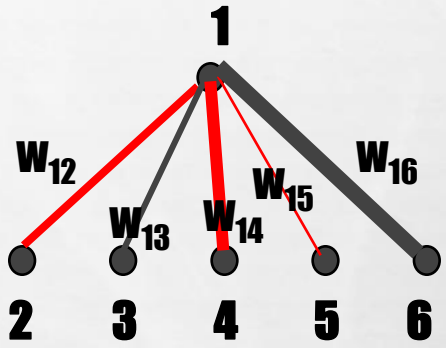
- Perceptron is considered a single-layer neural link with four main parameters.
- The perceptron model begins with multiplying all input values and their weights, then adds these values to create the weighted sum.
- Further, this weighted sum is applied to the activation function ‘f’ to obtain the desired output.
- This activation function is also known as the step function and is represented by ‘f.’

How Perceptron Work?



Perceptron

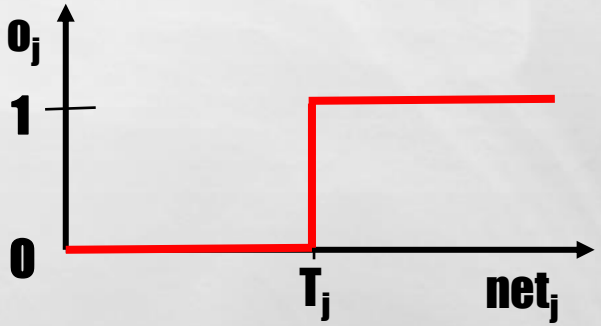
- Structure: single layer



- Non-linear output nodes

  Threshold units

$$o_j = \begin{cases} 0 \text{ if } net_j < T_j \\ 1 \text{ if } net_i \geq T_j \end{cases}$$

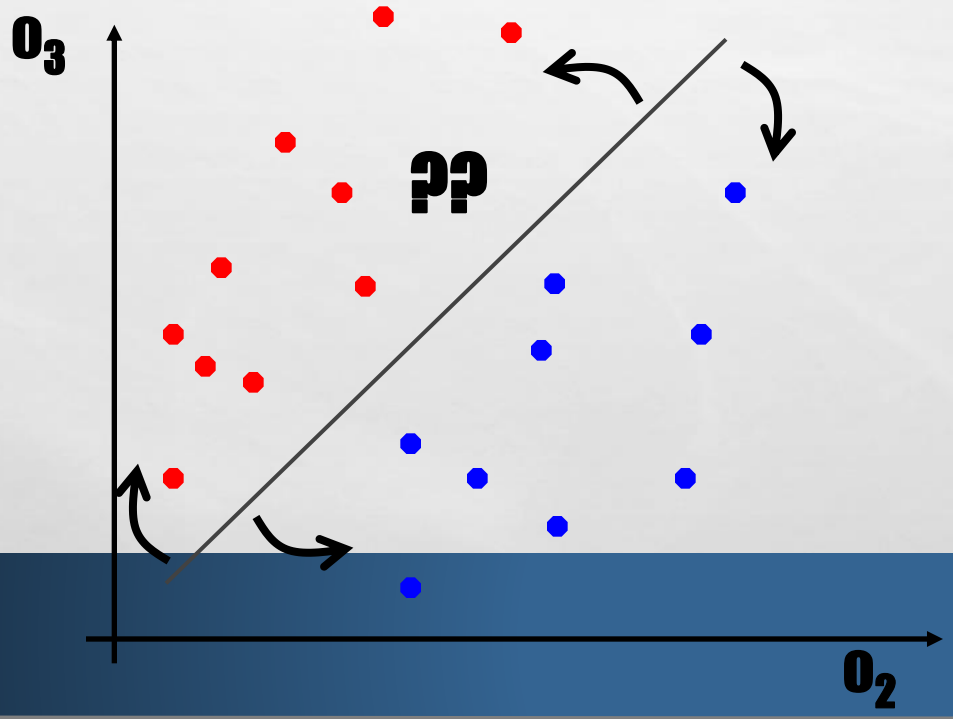


- Supervised learning
  - Learning weights so that output node produces correct output for each training sample

Perceptron as a Linear Separator

- Since perceptron uses linear threshold function, it is searching for a linear separator that discriminates the classes.

$$w_{12}o_2 + w_{13}o_3 > T_1$$

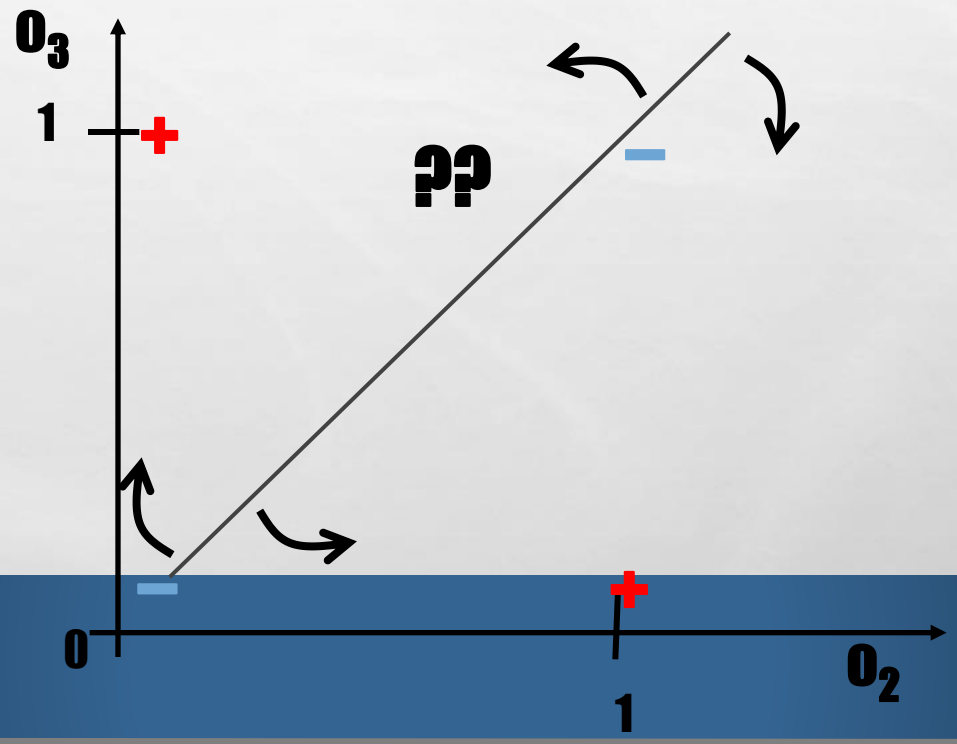


$$o_3 > -\frac{w_{12}}{w_{13}}o_2 + \frac{T_1}{w_{13}}$$

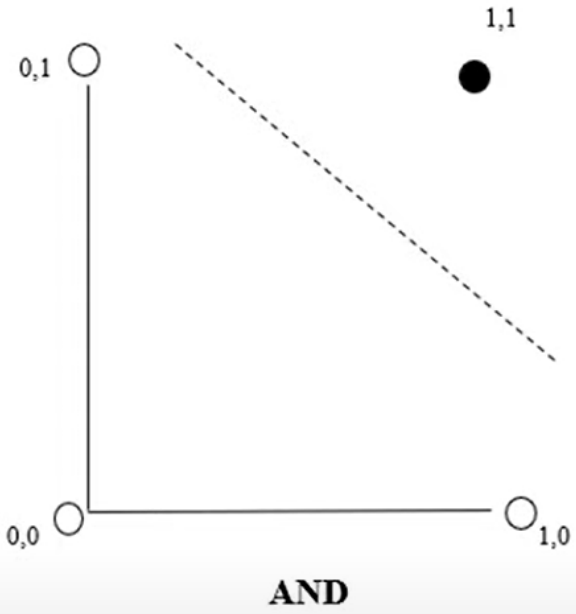
Or hyperplane in n-dimensional space

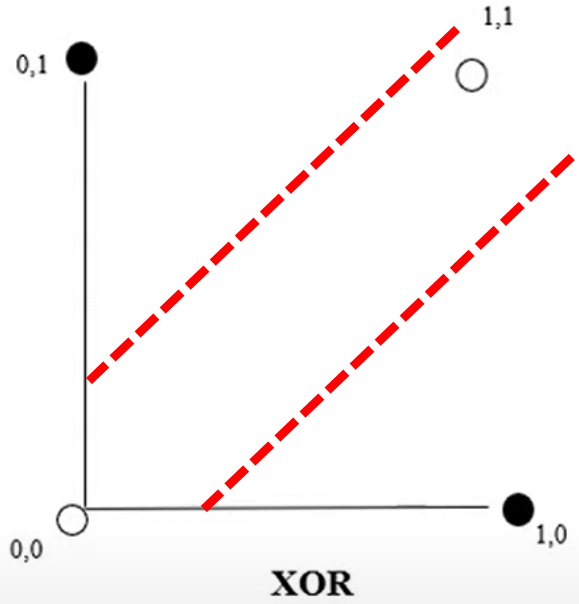
Concept Perceptron Cannot Learn

- Cannot learn exclusive-or, or parity function in general because they are not linearly separable.



What can perceptron represent





What can perceptron represent (cont.,)

- Functions which can be separated in this way **(AND function)** are called Linearly separable. Only linearly separable functions can be represented by perceptron.
- Thus perceptron can’t represent **(XOR function).**

Perceptron Convergence Theorem

- **Perceptron convergence theorem**: If the data is linearly separable and therefore a set of weights exist that are consistent with the data, then the Perceptron algorithm will eventually converge to a consistent set of weights.
- **Perceptron cycling theorem**: If the data is not linearly separable, the Perceptron algorithm will eventually repeat a set of weights and threshold at the end of some epoch and therefore enter an infinite loop.
  - By checking for repeated weights+threshold, one can guarantee termination with either a positive or negative result.

Perceptron Limits

- System obviously cannot learn concepts it cannot represent.
- Minksy and Papert (1969) wrote a book analyzing the perceptron and demonstrating many functions it could not learn.
- These results discouraged further research on neural nets; and symbolic AI became the dominate paradigm.

How Perceptron Work?

- The step function or Activation function is vital in ensuring that output is mapped between (0,1) or (-1,1).
- the weight of input indicates a node's strength.
- The bias 'b' is added to the weighted sum to improve the model's performance.
- Forward Stage: From the input layer in the on stage, activation functions begin and terminate on the output layer.
- Backward Stage: In the backward stage, **weight and bias values are modified per the model's requirement.** The backstage removed the error between the actual output and demands originating backward on the output layer.

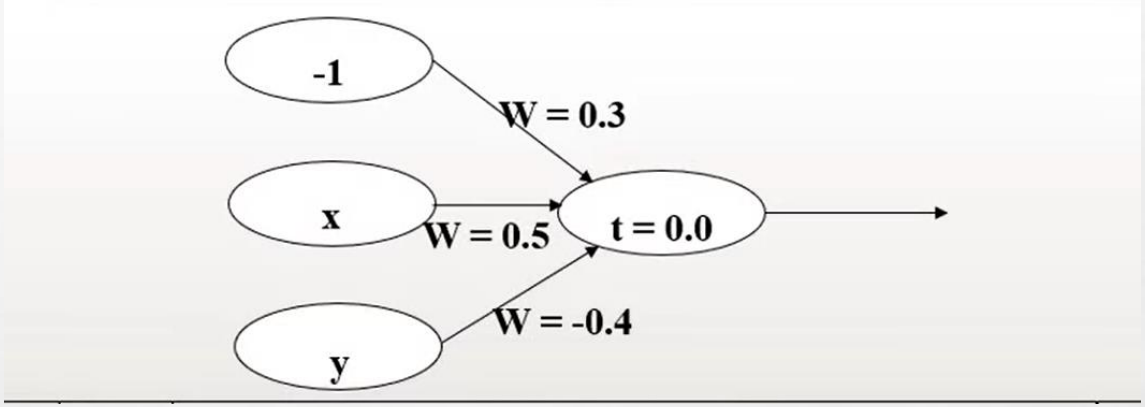
Perceptron Learning Algorithm

- Iteratively update weights until convergence.

  Initialize weights to random values
  Until outputs of all training examples are correct
  For each training pair, E, do:
  compute current output o_j for E given its inputs
  Compare current output to target value, t_j , for E
  Update synaptic weights using learning rule

- Each execution of the outer loop is typically called an ***epoch.***

Training a perceptron



I_1	I_2	I_3	Summation	Output
-1	0	0	(-1*.3)+(0*.5)+(0*-.4)= -0.3	0
-1	0	1	(-1*.3)+(0*.5)+(1*-.4)= -0.7	0
-1	1	0	(-1*.3)+(1*.5)+(0*-.4)= 0.2	1
-1	1	1	(-1*.3)+(1*.5)+(1*-.4)= -0.2	0

EPOCH

Perceptron Example

Imagine a perceptron (in your brain).

- The perceptron tries to decide if you should go to a concert.
- Is the artist good? Is the weather good?
- What weights should these facts have?

Perceptron Example

Criteria	Input	Weight
Artists is Good	**x1** = 0 or 1	**w1** = 0.7
Weather is Good	**x2** = 0 or 1	**w2** = 0.6
Friend will Come	**x3** = 0 or 1	**w3** = 0.5
Food is Served	**x4** = 0 or 1	**w4** = 0.3
Alcohol is Served	**x5** = 0 or 1	**w5** = 0.4

Threshold= 1.5

Multiply all inputs with its weights:

- x1 * w1 = 1 * 0.7 = 0.7
- x2 * w2 = 0 * 0.6 = 0
- x3 * w3 = 1 * 0.5 = 0.5
- x4 * w4 = 0 * 0.3 = 0
- x5 * w5 = 1 * 0.4 = 0.4

Sum all the results:

- 0.7 + 0 + 0.5 + 0 + 0.4 = 1.6 (The Weighted Sum)

Activate the Output:

- **Return true if the sum > 1.5 ("Yes I will go to the Concert")**

Limitation of Perceptron Model

- The output of a perceptron can only be a binary number (0 or 1) due to the hard-edge transfer function.
- It can only be used to classify the linearly separable sets of input vectors. If the input vectors are non-linear, it is not easy to classify them correctly.

Learning

- **Perceptron Learning Rule Algorithm Example.**
- **Delta Rule**

Perceptron learning rule algorithm example

X_1	X_2	t
0	0	0
0	1	1
1	0	1
1	1	1

Perceptron learning rule algorithm example

X1	X2	bias	w1	w2	W_bias	net	y	t
0	0	1						
0	1	1						
1	0	1						
1	1	1						

Perceptron learning rule algorithm example

X1	X2	bias	w1	w2	W_bias	net	y	t
0	0	1	0.1	0.2	-0.2			
0	1	1						
1	0	1						
1	1	1						

Activation function : step
threshold = .1

Perceptron learning rule algorithm example

X1	X2	bias	w1	w2	W_bias	net	y	t
0	0	1	0.1	0.2	-0.2	-0.2	0	0
0	1	1	0.1	0.2	-0.2	0	0	error 1
1	0	1	0.1	1.2	0.8	0.9	1	1
1	1	1	0.1	1.2	0.8	2.1	1	1

W(new)= w(old)+ (t-y) *x

Perceptron learning rule algorithm example

X1	X2	bias	w1	w2	W_bias	net	y	t
0	0	1	0.1	1.2	0.8	**0.8**	**1**	**0**
0	1	1	0.1	1.2	-0.2	1	1	1
1	0	1	0.1	1.2	-0.2	-0.1	**0**	**1**
1	1	1	0.1	1.2	0.8	3.1	1	1

Error

Perceptron learning rule algorithm example

x1	x2	bias	w1	w2	w_bias	net	y	t
0	0	1	1.1	1.2	-0.2	-0.2	0	0
0	1	1	1.1	1.2	-0.2	1	1	1
1	0	1	1.1	1.2	-0.2	0.9	1	1
1	1	1	1.1	1.2	-0.2	2.1	1	1

Perceptron Learning Rule (delta rule)

- Update weights by: $w_{ji} = w_{ji} + \eta(t_j - o_j)o_i$

  Where η is the "learning rate"

  T_j is the teacher specified output for unit j

  $(T_j - o_j)$ is the **error** (training is error driven)

- Equivalent to rules:
  - If output is correct do nothing.
  - If output is high, lower weights on active inputs
  - If output is low, increase weights on active inputs

(Delta Rule) example

X1	X2	bias	w1	w2	W_bias	net	y	t
0	0	1	0.1	0.2	-0.2			
0	1	1						
1	0	1						
1	1	1						

Activation function : step, threshold = .1

Assume learning rate = .1

(Delta Rule) example

x1	x2	bias	w1	w2	wb	net	y	t
0	0	1	0.1	0.2	-0.2	-0.2	0	0
0	1	1	0.1	0.2	-0.2	0	0	1
1	0	1	0.1	0.3	-0.1			1
1	1	1						1

update weights:

w1 := w1 + 0.1 * x1 * (t - y) → w1 := 0.1 + 0.1 * 0 * 1 → 0.1
w2 := w2 + 0.1 * x2 * (t - y) → w2 := 0.2 + 0.1 * 1 * 1 → 0.3
wb := wb + 0.1 * bias * (t - y) → wb := -0.2 + 0.1 * 1 * 1 → -0.1

(Delta Rule) example

x1	x2	bias	w1	w2	wb	net	y	t
0	0	1	0.1	0.2	-0.2	-0.2	0	0
0	1	1	0.1	0.2	-0.2	0	0	1
1	0	1	0.1	0.3	-0.1	0	0	1
1	1	1	0.2	0.3	0			1

update weights:

w1 := w1 + 0.1 * x1 * (t - y)	→	0.1 + 0.1 * 1 * 1	→ 0.2
w2 := w2 + 0.1 * x2 * (t - y)	→	0.3 + 0.1 * 0 * 1	→ 0.3
wb := wb + 0.1 * bias * (t - y)	→	-0.1 + 0.1 * 1 * 1	→ 0

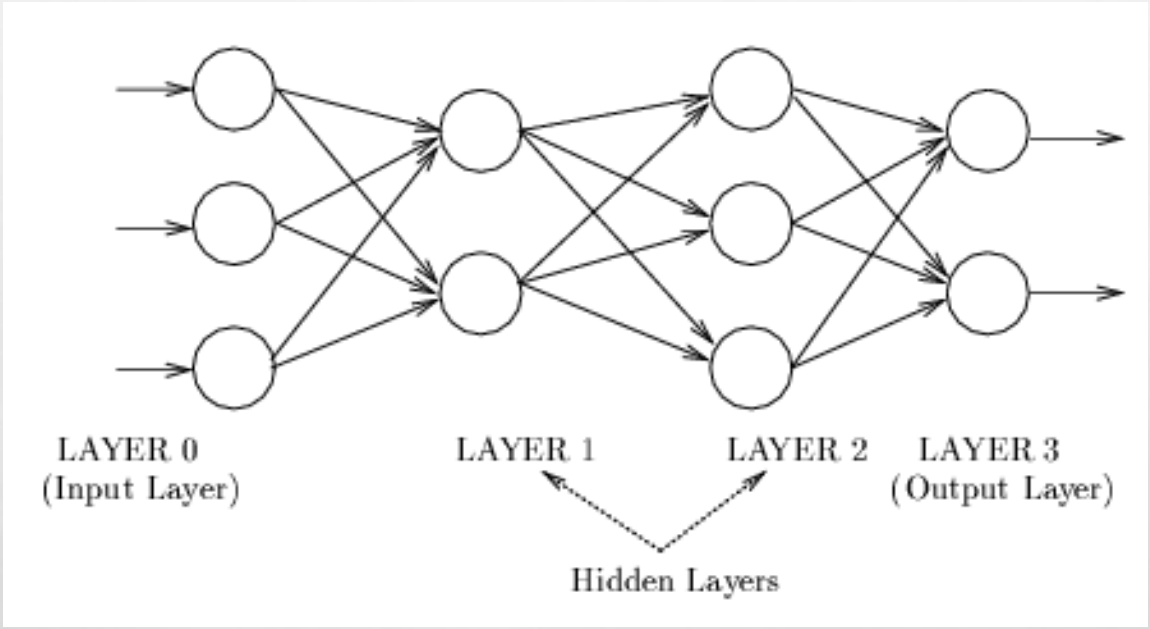
(Delta Rule) example

x1	x2	bias	w1	w2	wb	net	y	t
0	0	1	0.2	0.3	0	0	**0**	**0**
0	1	1	0.2	0.3	0	0.3	**1**	**1**
1	0	1	0.2	0.3	0	0.2	**1**	**1**
1	1	1	0.2	0.3	0	0.5	**1**	**1**
Weights for next epoch:			0.2	0.3	0			

Second epoch has no errors → stop training

Feedforward networks

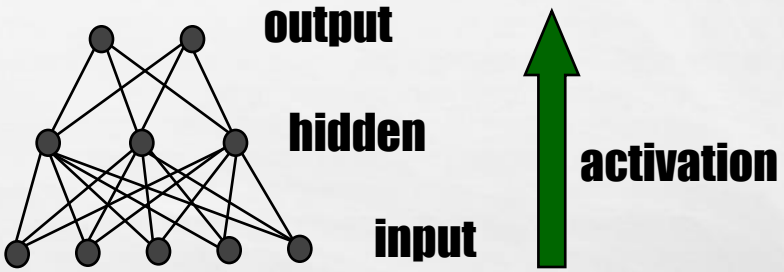
- A connection is allowed from a node in layer *i* only to nodes in layer *i* + 1.
- Most widely used architecture.



Conceptually, nodes at higher levels successively abstract features from preceding layers

Multi-Layer Feed-Forward Networks

- A typical multi-layer network consists of an input layer, one or more hidden and output layer, each fully connected to the next, with activation feeding forward.



- **Nodes at hidden layers MUST be non-linear** (typically a sigmoid function)
- Given an arbitrary number of hidden units with a single hidden layer, there exists a set of weights which can compute any given Boolean function (or any L2 function). But an effective learning algorithm for such networks was thought to be difficult.

Gradient Descent

- Define objective to minimize error:

$$E(W) = \sum_{d \in D} \sum_{k \in K} (t_{kd} - o_{kd})^2$$

where D is the set of training examples, K is the set of output units, t_{kd} and o_{kd} are, respectively, the teacher and current output for unit k for example d.

- The derivative of a sigmoid unit with respect to net input is:

$$\frac{\partial o_j}{\partial net_j} = o_j(1 - o_j)$$

- Learning rule to change weights to minimize error is:

$$\Delta w_{ji} = -\eta \frac{\partial E}{\partial w_{ji}}$$

Backpropagation Learning Rule

- Each weight changed by:

$$\Delta w_{ji} = \eta \delta_j o_i$$

$$\delta_j = o_j(1-o_j)(t_j - o_j) \qquad \text{if } j \text{ is an output unit}$$

$$\delta_j = o_j(1-o_j)\sum_k \delta_k w_{kj} \qquad \text{if } j \text{ is a hidden unit}$$

where η is a constant called the learning rate

t_j is the correct teacher output for unit j

δ_j is the error measure for unit j

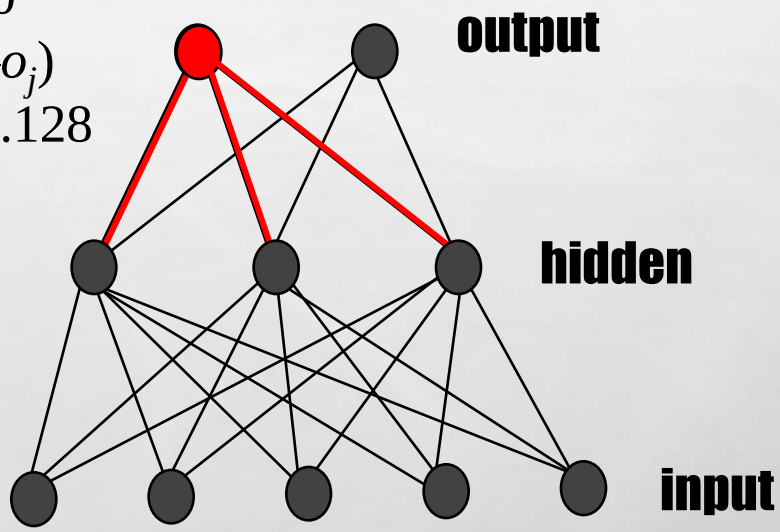
Error Backpropagation

- First calculate error of output units and use this to change the top layer of weights.

Current output: o_j=0.2
Correct output: t_j=1.0
Error $\delta_j = o_j(1–o_j)(t_j–o_j)$
0.2(1–0.2)(1–0.2)=0.128

Update weights into *j*

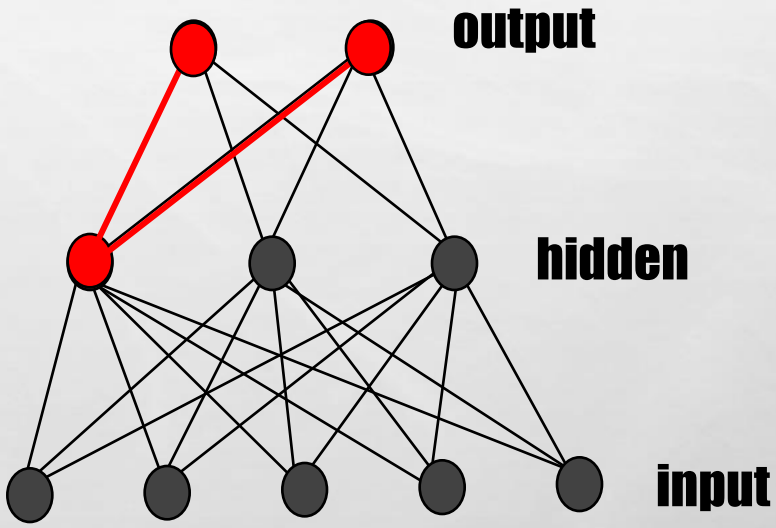
$$\Delta w_{ji} = \eta \delta_j o_i$$



Error Backpropagation

- Next calculate error for hidden units based on errors on the output units it feeds into.

$$\delta_j = o_j(1-o_j)\sum_k \delta_k w_{kj}$$



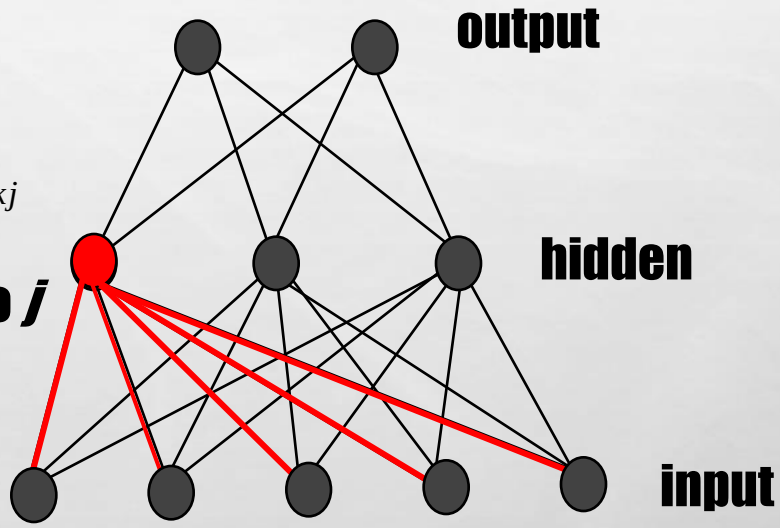
Error Backpropagation

- Finally update bottom layer of weights based on errors calculated for hidden units.

$$\delta_j = o_j(1-o_j)\sum_k \delta_k w_{kj}$$

Update weights into *j*

$$\Delta w_{ji} = \eta \delta_j o_i$$



Backpropagation Training Algorithm

Create the 3-layer network with *H* hidden units with full connectivity between layers. Set weights to small random real values.

Until all training examples produce the correct value (within ε), or mean squared error ceases to decrease, or other termination criteria:

- Begin epoch
- For each training example, *d*, do:
  - Calculate network output for *d*'s input values
  - Compute error between current output and correct output for *d*
  - Update weights by backpropagating error and using learning rule
- End epoch

Hidden Unit Representations

- Trained hidden units can be seen as newly constructed features that make the target concept linearly separable in the transformed space.
- On many real domains, hidden units can be interpreted as representing meaningful features such as vowel detectors or edge detectors, etc..
- However, the hidden layer can also become a distributed representation of the input in which each individual unit is not easily interpretable as a meaningful feature.

Successful Applications

- Text to Speech (NetTalk)
- Fraud detection
- Financial Applications
  - HNC (eventually bought by Fair Isaac)
- Chemical Plant Control
  - Pavillion Technologies
- Automated Vehicles
- Game Playing
  - Neurogammon
- Handwriting recognition

Hand-written Character Recognition

- MNIST: a data set of hand-written digits
  - 60,000 training samples
  - 10,000 test samples
  - Each sample consists of 28 x 28 = 784 pixels
- Various techniques have been tried
  - Linear classifier: 12.0%
  - 2-layer BP net (300 hidden nodes) 4.7%
  - 3-layer BP net (300+200 hidden nodes) 3.05%
  - Support vector machine (SVM) 1.4%
  - Convolutional net 0.4%
  - **6 layer BP net (7500 hidden nodes): 0.35%**

Failure rate for test samples

Strengths of BP Learning

- **Great representation power**
  - Any meaningful function can be represented by a BP net
  - Many such functions can be approximated by BP learning (gradient descent approach)
- **Easy to apply**
  - Only requires a good set of training samples
  - Does not require substantial prior knowledge or deep understanding of the domain itself (ill structured problems)
  - Tolerates noise and missing data in training samples (graceful degrading)
- **Easy to implement** the core of the learning algorithm
- **Good generalization power**
  - Often produces accurate results for inputs outside the training set

Deficiencies of BP Learning

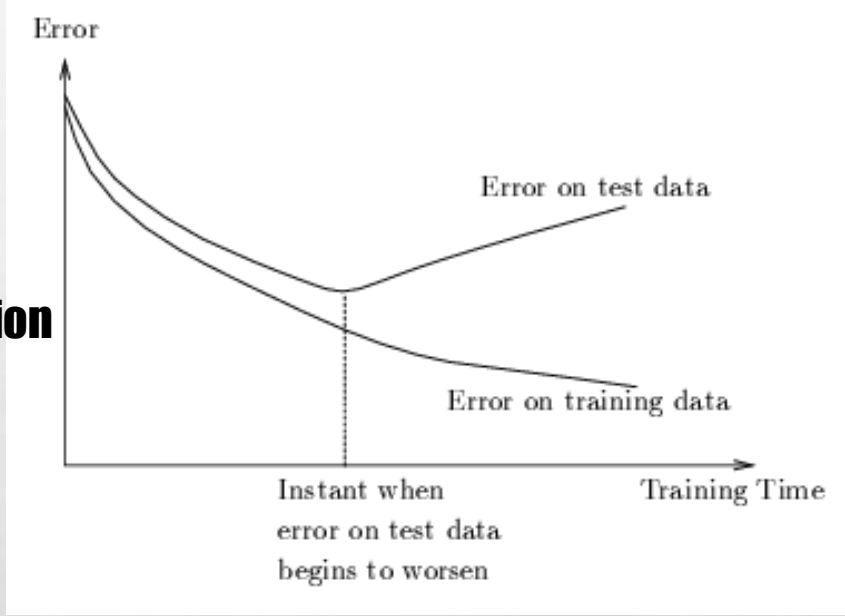
- Learning often takes a **long time** to converge
- Complex functions often need hundreds or thousands of epochs
- The network is essentially a **black box**
  - It may provide a desired mapping between input and output vectors (***x***, ***o***) but does not have the information of why a particular ***x*** is mapped to a particular ***o***.
  - It thus cannot provide an intuitive (e.g., causal) explanation for the computed result.
  - This is because the hidden nodes and the learned weights do not have clear semantics.
    - What can be learned are operational parameters, not general, abstract knowledge of a domain
  - Unlike many statistical methods, there is no theoretically well-founded way to **assess the quality** of BP learning
    - What is the confidence level of *o* computed from input *x* using such net?
    - What is the confidence level for a trained BP net, with the final training/test error (which may or may not be close to zero)?

- Problem with gradient descent approach
  - only guarantees to reduce the total error to a **local minimum**. (***E*** might not be reduced to zero)
    - Cannot escape from the local minimum error state
    - **Not every function that is representable can be learned**
  - How bad: depends on the shape of the error surface. Too many valleys/wells will make it easy to be trapped in local minima
  - Possible remedies:
    - Try nets with different # of hidden layers and hidden nodes (they may lead to different error surfaces, some might be better than others)
    - Try different initial weights (different starting points on the surface)
    - Forced escape from local minima by random perturbation (e.g., **simulated annealing**)

- **Generalization** is not guaranteed even if the error is reduced to 0
  - **Over-fitting**/over-training problem: trained net fits the training samples perfectly (*E* reduced to 0) but it does not give accurate outputs for inputs not in the training set

– Possible remedies:
  - More and better samples
  - Using smaller net if possible
  - Using larger error bound (forced early termination)
  - Introducing noise into samples
    – modify $(x_1,\ldots, x_n)$ to $(x_1+\alpha_1,\ldots, x_n+\alpha_n)$ where α_i are small random displacements

Scaling function



  - **Cross-Validation**
    – leave some (~20%) samples as test data (not used for weight update)
    – periodically check error on test data
    – learning stops when error on test data starts to increase

Assignment

- Define the different types of activation function, and mention when to use each one according to its pros and cons?

Review Questions

- Explain the components of the artificial neural networks, and describe how it works?
- Differentiate between feedforward neural networks and recurrent neural networks?
- Differentiate between feedforward backpropagation in neural networks?
- What is the main purpose of the activation function?
- Explain the Hebbian learning.
- Explain the perceptron neural network.
- Why perceptron Cannot learn exclusive-or?

Review Questions

1. How back propagation neural networks are being learned or “ trained”?
2. What is the Gradient Descent algorithm and how it used, mention pros and cons.
3. What is Hebbian Learning in ANN?
4. What is the Cons and pros of back propagation neural network?

Review Questions

Based on the perceptron Learning algorithm, **find the number of epochs** that are needed to learn the following :

- **the logical and operation.**
- **the logical OR operation**

X1	X2	bias	w1	w2	W_bias	net	y	t
0	0	1	0.1	0.2	-0.2			
0	1	1						
1	0	1						
1	1	1						

Activation function: step, threshold = .1

Review Questions

- Based on the delta rule algorithm, **find the number of epochs** that are needed to learn the **neural network the logical and operation.**
- Based on the delta rule algorithm, **find the number of epochs** that are needed to learn the **neural network the logical OR operation**

X1	X2	bias	w1	w2	W_bias	net	y	t
0	0	1	0.1	0.2	-0.2			
0	1	1						
1	0	1						
1	1	1						

Activation function: step, threshold = .1, learning rate .1

Questions ?

Thank You